https://hjfy.top/arxiv/2310.08560

MemGPT: 面向大语言模型的操作系统

Charles Packer [1] **Sarah Wooders** [1] **Kevin Lin** [1]
Vivian Fang [1] **Shishir G. Patil** [1] **Ion Stoica** [1] **Joseph E. Gonzalez** [1]

Abstract

大语言模型（LLMs）已彻底改变了人工智能领域，但其在扩展对话和文档分析等任务中的应用受到有限上下文窗口的制约。为突破这一限制，我们提出虚拟上下文管理，该技术受传统操作系统中分层内存系统的启发，通过物理内存与磁盘之间的分页机制，实现对扩展虚拟内存的“假象”。利用该技术，我们引入 MemGPT (MemoryGPT)，一个能够智能管理不同存储层级的系统，从而在 LLM 有限的上下文窗口内有效提供扩展上下文。我们在两个领域评估了这一受操作系统启发的设计：在文档分析领域，MemGPT 能够处理远超底层 LLM 上下文窗口容量的大规模文档；在多轮会话场景中，MemGPT 可构建能够记忆、反思并随长期用户交互动态演化的会话智能体。我们已在https://research.memgpt.ai 发布 MemGPT 的代码与实验数据。

1. 引言

近年来，大模型（LLMs）及其潜在的 Transformer 架构 (Vaswani et al., 2017; Devlin et al., 2018; Brown et al., 2020; Ouyang et al., 2022) 已成为对话式人工智能的基石，并催生了众多面向消费者和企业端的应用。尽管取得了这些进展，大模型所使用的有限固定长度上下文窗口显著限制了其在长对话或长文档推理方面的适用性。例如，目前最广泛使用的开源大模型仅能支持几十轮的来回消息交互，或对短文档进行推理，一旦超过其最大输入长度 (Touvron et al., 2023) 就会失效。

直接扩展 Transformer 的上下文长度会由于 Transformer 架构中的自注意力机制导致计算时间和内存成本呈二次增长，使得设计新型长上下文架构成为一项紧迫的研究挑战 (Dai et al., 2019; Kitaev et al., 2020; Beltagy et al., 2020)。尽管开发更长模型是当前研究的活跃领域 (Dong et al., 2023)，即使我们能够克服上下文缩放带来的计算挑战，最近的研究表明，长上下文模型难以有效利用额外的上下文信息 (Liu et al., 2023a)。因此，在训练先进大模型所需资源巨大且上下文缩放收益递减的情况下，迫切需要替代技术来支持长上下文。

在本文中，我们研究了如何在继续使用固定上下文模型的同时，提供无限上下文的假象。我们的方法借鉴了虚拟内存分页的思想，该思想通过在主内存和磁盘之间分页数据，使应用程序能够处理远超可用内存的数据集。我们利用大语言模型智能体在函数调用能力方面的最新进展 (Schick et al., 2023; Liu et al., 2023b)，设计了 MemGPT，一个受操作系统启发的大语言模型系统，用于 **虚拟上下文管理**。通过函数调用，大语言模型智能体可以读写外部数据源，修改自身的上下文，并自主选择何时向用户返回结果。

这些能力使大模型能够有效地在上下文窗口（类比于操作系统的“主内存”）与外部存储之间进行信息的“分页”操作，类似于传统操作系统中的层次化内存。此外，可以通过调用函数来管理上下文管

[1]University of California, Berkeley. Correspondence to: Charles Packer <cpacker@berkeley.edu>.

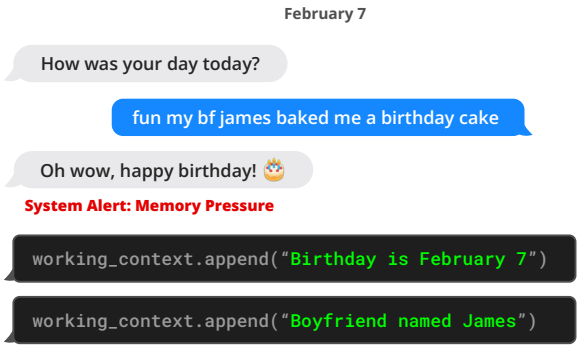

Figure 1. MemGPT (left) 在接收到关于上下文空间受限的系统警报后，将数据写入持久内存。

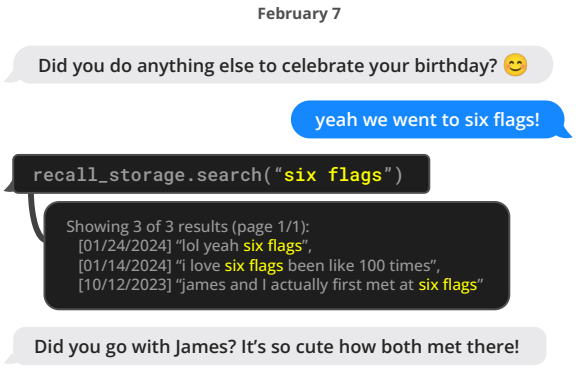

Figure 2. MemGPT（左）可以搜索上下文外的数据，将相关信息引入当前的上下文窗口。

理、响应生成和用户交互之间的控制流。这使得智能体能够选择对单个任务的上下文内容进行迭代修改，从而更有效地利用其有限的上下文。

在 MemGPT 中，我们将上下文窗口视为一种受限的内存资源，并为大模型设计了一种类比于传统操作系统中内存层级的内存层次结构 (Patterson et al., 1988)。传统操作系统中的应用程序与虚拟内存交互，操作系统通过将溢出的数据分页到磁盘，并在应用程序访问时通过页错误将其重新调回内存，从而提供比实际物理（即主）内存更多的内存资源的假象。为了实现类似更长上下文长度的假象（类比于虚拟内存），我们允许大模型通过一个名为 "LLM OS" 的系统自主管理其自身上下文中的内容（类比于物理内存），该系统即为 MemGPT 。MemGPT 使得大模型能够检索当前上下文中缺失的相关历史数据，同时将较不相关的数据从上下文中移出并存储至外部存储系统。图 3 展示了 MemGPT 的组成部分。

通过结合内存层次结构、操作系统功能和基于事件的控制流，MemGPT 能够利用具有有限上下文窗口的大模型处理无限上下文。为展示我们新型受操作系统启发的大模型系统的优势，我们在两个现有大模型因上下文容量有限而性能严重受限的领域进行了评估：文档分析领域，标准文本文件的长度可能迅速超过现代大模型的输入容量；会话智能体领域，受限于有限对话窗口的大模型在长时间对话中缺乏情境感知能力、角色一致性以及长期记忆。在这两种情景下，MemGPT 均能够克服有限上下文的限制，优于现有的基于大模型的方法。

2. MemGPT (MemoryGPT)

MemGPT 的受操作系统启发的多级内存架构区分了两种主要的内存类型：**主上下文**（类比为主存/物理内存/RAM）和**外部上下文**（类比为磁盘内存/磁盘存储）。主上下文由大模型的提示 *token* 组成——主上下文中的任何内容均被视为在上下文中，可在推理过程中被大模型处理器访问。外部上下文指位于大模型固定上下文窗口之外的任何信息。这些超出上下文的数据必须始终显式地移入主上下文，才能在推理过程中传递给大模型处理器。MemGPT 提供了函数调用，使大模型处理器能够自主管理其内存，而无需用户干预。

2.1. 主要上下文（*prompt tokens*）

MemGPT 中的 prompt token 被划分为三个连续的部分：**系统指令**、**工作上下文**和 **FIFO 队列**。系统指令为只读（静态）内容，包含 MemGPT 控制流程信息、不同内存层级的预期用途，以及如何使用 MemGPT 功能的说明（例如，如何检索上下文外的数据）。工作上下文是一个固定大小的可读写块，其内容为非结构化文本，仅可通过 MemGPT 函数调用进行写入。在对话情景中，工作上下文旨在存储用户的关键事实、偏好以及其他关于用户和智能体所采用人设的重要信息，从而帮助智能体与用户流畅对话。

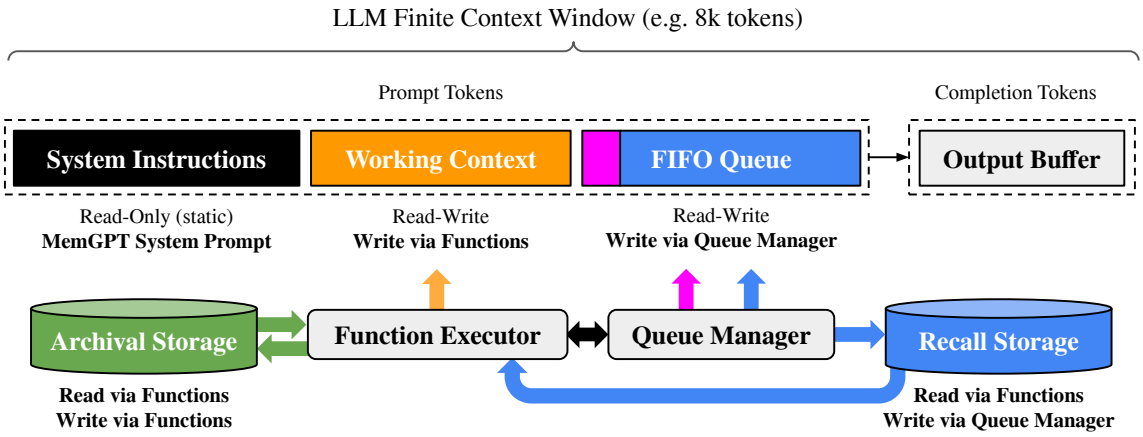

Figure 3. 在 MemGPT 中，一个固定上下文的 LLM 处理器通过引入分层记忆系统以及一系列功能得以增强，使其能够自主管理自身记忆。LLM 的提示 token（输入），或称 *主上下文*，由系统指令、工作上下文和一个先进先出队列组成。LLM 的完成 token（输出）由函数执行器解释为函数调用。MemGPT 使用函数在主上下文与 *外部上下文*（归档和召回存储数据库）之间移动数据。LLM 可通过在其输出中生成特殊关键字参数（`request_heartbeat=true`）请求立即后续的 LLM 推理，从而将多个函数调用串联起来；这种函数链机制使得 MemGPT 能够执行多步检索以回答用户查询。

FIFO 队列用于存储滚动的消息历史，包括智能体与用户之间的消息，以及系统消息（例如，内存警告）和函数调用的输入输出。FIFO 队列的第一个索引存储一条系统消息，其中包含已被从队列中淘汰的消息的递归摘要。

2.2. 队列管理器

消息队列管理器负责管理 **召回存储**和 **FIFO 队列**中的消息。当系统接收到新消息时，消息队列管理器将传入的消息追加到 FIFO 队列中，连结提示 token 并触发大语言模型推理以生成大语言模型输出（即完成 token）。消息队列管理器将传入的消息和生成的大语言模型输出同时写入召回存储（即 MemGPT 消息数据库）。当通过 MemGPT 函数调用检索召回存储中的消息时，消息队列管理器会将这些消息追加到队列末尾，以便重新插入到大语言模型的上下文窗口中。

队列管理器还负责通过队列驱逐策略控制上下文上溢。当提示 token 数量超过底层 LLM 上下文窗口的‘警告 token 数量‘（例如上下文窗口的 70%）时，队列管理器会向队列中插入一条系统消息，警告 LLM 即将发生队列驱逐（即‘内存压力‘ 警告），以便 LLM 能够使用 MemGPT 函数将先进先出队列中包含的重要信息存储到工作上下文中或 **归档存储**（一个可存储任意长度文本对象的读/写数据库）。当提示 token 数量超过‘刷新 token 数量‘（例如上下文窗口的 100%）时，队列管理器会刷新队列以释放上下文窗口中的空间：队列管理器驱逐特定数量的消息（例如上下文窗口的 50%），并使用现有的递归摘要和被驱逐的消息生成新的递归摘要。队列刷新后，被驱逐的消息将不再处于上下文中，也无法立即被 LLM 查看，但它们会被无限期地存储在召回存储中，并可通过 MemGPT 函数调用读取。

2.3. 函数执行器（*completion token* 的处理）

MemGPT 通过由 LLM 处理器生成的函数调用，在主上下文与外部上下文之间协调数据流动。记忆的编辑与检索完全由自身主导：MemGPT 能够根据当前上下文自主更新并搜索其自身记忆。例如，它可以决定何时在不同上下文间移动项目（例如，当对话历史变得过长时，如图 1 所示），并修改其主上下文以更好地反映其对当前目标和职责的动态理解（如图 3 所示）。我们通过在系统指令中提供明确的指示来实现自主导的编辑与检索，这些指示指导 LLM 如何与 MemGPT 记忆系统交互。这些指示包含两个主要部分：(1) 对记忆层级及其各自用途的详细描述，以及 (2) 一个函数模式（附带其自然语

Table 1. 比较常用模型和大语言模型 API 的上下文长度(数据收集于 2024 年 1 月)。* 假设预提示（preprompt）为 1k token，平均消息大小为~50 token（~250 字符）的消息数量估算。'开源' 表示该模型为开源或开放权重（而非仅通过 API 提供）。

Model / API name	Open?	Context Window Tokens	*Messages
Llama (1)	✓	2k	20
Llama 2	✓	4k	60
GPT-3.5 Turbo (release)	✗	4k	60
Mistral 7B	✓	8k	140
GPT-4 (release)	✗	8k	140
GPT-3.5 Turbo	✗	16k	300
GPT-4	✗	32k	~600
Claude 2	✗	100k	~2000
GPT-4 Turbo	✗	128k	~2600
Yi-34B-200k	✓	200k	~4000

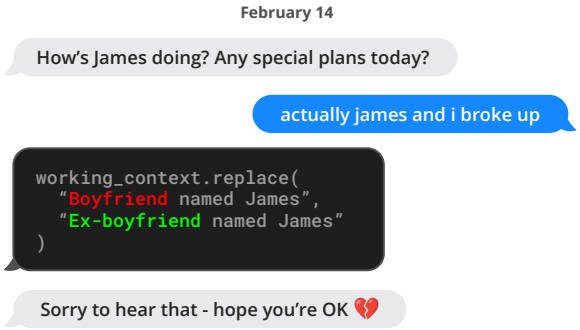

Figure 4. 一个示例对话片段，其中 MemGPT （左侧）更新了存储的信息。此处信息存储在工作上下文内存中（位于提示 token 内）。

言描述)，系统可通过该模式调用以访问或修改其记忆。

在每次推理循环中，LLM 处理器将主上下文（连结成单一字符串）作为输入，并生成一个输出字符串。该输出字符串由 MemGPT 解析以确保正确性，若解析器验证函数参数有效，则执行该函数。结果（包括可能发生的运行时错误，例如当主上下文已达到最大容量时仍尝试添加内容）随后由 MemGPT 反馈给处理器。这一反馈环使系统能够从其动作中学习并相应地调整行为。对上下文限制的意识是使自编辑机制有效运作的关键方面；为此，MemGPT 会向处理器提示有关 token 限制的警告，以指导其内存管理决策。此外，我们的记忆检索机制被设计为充分考虑这些 token 约束，并实现分页机制，以防止检索调用导致上下文窗口上溢。

2.4. 控制流和函数链

在 MemGPT 中，事件触发大语言模型推理：事件是 MemGPT 的泛化输入，可包含用户消息（在聊天应用中)、系统消息（例如主上下文容量警告)、用户交互（例如用户刚刚登录的提示，或文档上传完成的提示)，以及定期运行的定时事件（使 MemGPT 能够在无需用户干预的情况下 "自发" 运行)。MemGPT 使用解析器处理事件，将其转换为纯文本消息，这些消息可被追加到主上下文中，并最终作为输入送入大语言模型处理器。

许多实际任务需要按顺序调用多个函数，例如从单个查询中导航到多页结果，或从不同的文档中收集数据并整合到主上下文中。函数链式调用允许 MemGPT 在返回控制权给用户之前，按顺序执行多个函数调用。在 MemGPT 中，可以通过一个特殊标志来调用函数，该标志请求在指定函数执行完成后立即将控制权返回给处理器。如果该标志存在，MemGPT 将把函数输出添加到主上下文中，并（与暂停处理器执行相反）继续执行。如果该标志不存在（即为 *yield*)，MemGPT 将不会运行 LLM 处理器，直到下一个外部事件触发（例如用户消息或定时中断)。

3. 实验

我们在两个长上下文领域对 MemGPT 进行评估：会话智能体和文档分析。对于会话智能体，我们扩展了现有的多会话对话数据集 (Xu et al., 2021)，并引入了两个新的对话任务，用以评估智能体在长时间对话中保持知识的能力。对于文档分析，我们在 (Liu et al., 2023a) 提供的现有任务上对 MemGPT 进行基准测试，涵盖长文档上的问答系统和键值检索。此外，我们提出了一项新的嵌套键值检索任务，该任务需要跨多个数据源整合信息，从而测试智能体从多个数据源中进行多跳检索的能力。我们已公

开发布增强版的 MSC 数据集、嵌套键值检索数据集，以及包含 2000 万篇维基百科文章嵌入的数据集，以促进未来的研究。我们的基准测试代码可在 https://research.memgpt.ai 获取。

实现细节。在讨论 OpenAI 模型时，除非另有说明，'GPT-4 Turbo' 指的是特定的 `gpt-4-1106-preview` 模型接口（上下文窗口为 128,000），'GPT-4' 指的是 `gpt-4-0613`（上下文窗口为 8,192），而 'GPT-3.5 Turbo' 指的是 `gpt-3.5-turbo-1106`（上下文窗口为 16,385）。在实验中，我们使用所有基准模型（GPT-4、GPT-4 Turbo 和 GPT 3.5）运行 MemGPT，以展示潜在模型性能如何影响 MemGPT 的表现。

Table 2. **深度记忆检索（DMR）性能**。在此任务中，智能体被询问关于先前对话（会话 1–5）中讨论的主题的具体问题。智能体的回答将根据标准答案进行评分。MemGPT 显著优于固定上下文的基准方法。

Model	Accuracy ⇑	ROUGE-L (R) ⇑
GPT-3.5 Turbo	38.7%	0.394
+ MemGPT	66.9%	0.629
GPT-4	32.1%	0.296
+ MemGPT	92.5%	0.814
GPT-4 Turbo	35.3%	0.359
+ MemGPT	**93.4%**	**0.827**

3.1. MemGPT 用于会话智能体

会话智能体（如虚拟伙伴和个性化助手）旨在与用户展开自然、长期的互动，持续时间可能长达数周、数月甚至数年。这给具有固定长度上下文的模型带来了挑战，因为它们只能参考有限的对话历史。一个 "无限上下文" 智能体应能无缝处理连续交流，而无需边界或重置。在与用户对话时，此类智能体必须满足两个关键标准：(1) 一致性——智能体应保持对话连贯性。新出现的事实、偏好和事件应与用户及智能体此前的声明保持一致。(2) 参与度——智能体应利用对用户的长期知识来个性化回应。引用之前的对话内容可使对话更加自然且更具吸引力。

因此，我们从以下两个标准评估所提出的系统 MemGPT：(1) MemGPT 是否利用其记忆来提升对话的一致性？它能否记住过去交互中的相关事实、偏好和事件以维持连贯性？(2) MemGPT 是否通过利用记忆产生更具吸引力的对话？它是否能自发地融入长期用户信息以个性化消息？通过在一致性和参与度上的评估，我们可以判断 MemGPT 在处理长期会话互动挑战方面相较于固定上下文基准的表现。其满足这些标准的能力将表明，无界上下文是否为会话智能体带来实际益处。

数据集。我们在 Xu et al. (2021) 提出的多轮会话聊天（MSC）数据集上评估了 MemGPT 和我们的固定上下文基准方法，该数据集包含由人工标注者生成的多轮会话日志，每位标注者需在所有会话期间保持一致的人设。MSC 中的每个多轮会话共包含五个会话，每个会话包含约十几个消息。在一致性实验中，我们新增了一个会话（第 6 会话），其中包含相同两个人设之间的一对问答响应。

3.1.1. 深度记忆检索任务（一致性）。

我们基于 MSC 数据集提出了一项新的 "深度记忆检索"（DMR）任务，旨在测试会话智能体的一致性。在 DMR 任务中，用户提出的问题明确指向先前的对话内容，且预期答案的值域非常狭窄。我们使用一个分离的大型语言模型生成了 DMR 问答对，该模型被指令编写一个用户向另一个用户提出的问题，该问题只能通过利用过往对话中获得的知识才能正确回答（详见附录获取更多细节）。

我们使用 ROUGE-L 得分 (Lin, 2004) 和一个 "LLM 判定器" 来评估生成响应与 'gold response' 之间的质量，该判定器被指示判断生成的响应是否与黄金响应一致（已有研究表明 GPT-4 与人类评估者具有高度一致性 (Zheng et al., 2023)）。在实际操作中，我们注意到生成的响应（来自 MemGPT 和基线方法）通常比黄金响应更为冗长。因此，我们采用 ROUGE-L 召回率（R）指标，以考虑生成智能体回复相较于相对简短的黄金答案标签所表现出的

Table 3. **对话开场白性能**。智能体的对话开场白通过与标准人格标签（SIM-1/3）以及人类创作的开场白（SIM-H）的相似度得分进行评估。MemGPT 在多种潜在模型下均能够超越人类创作的对话开场白性能。

Method	⇑ SIM-1	SIM-3	SIM-H
Human	0.800	0.800	1.000
GPT-3.5 Turbo	0.830	0.812	**0.817**
GPT-4	**0.868**	**0.843**	0.773
GPT-4 Turbo	0.857	0.828	0.767

冗长度。

MemGPT 利用记忆来维持连贯性：表 2 展示了 MemGPT 与固定内存基准方法的性能对比。我们比较了使用不同潜在大模型的 MemGPT，同时也将其与未使用 MemGPT 的基础大模型作为基准进行对比。基准方法仅能查看过去五轮对话的有损摘要，以模拟扩展的递归摘要过程，而 MemGPT 则能够访问完整的对话历史，但必须通过分页搜索查询来召回记忆（以便将其引入主上下文）。在此任务中，我们观察到 MemGPT 显著提升了潜在基础大模型的性能：从 MemGPT 转换到相应的基础大模型时，准确率和 ROUGE 得分均明显下降。

3.1.2. 对话开场白任务（参与度）。

在“对话开场白”任务中，我们评估智能体根据先前对话中积累的知识，创作吸引用户注意力的开场消息的能力。为了使用 MSC 数据集评估对话开场白的“吸引力”，我们将生成的开场白与标准人格档案（gold personas）进行比较：一个具有吸引力的开场白应当引用人格档案中包含的一个（或多个）数据点，而这些数据点在 MSC 中有效地总结了所有先前会话中积累的知识。我们还将其与人工生成的标准开场白进行比较，即后续会话中的首个回复。我们在表 3中报告了 MemGPT 生成开场白的 CSIM 得分。我们测试了使用不同基础大模型的 MemGPT 的多种变体。

MemGPT 利用记忆来提升参与度：如表 3 所示，MemGPT 能够生成与人工编写的开场白表现相当甚至偶尔更优的吸引人开场白。我们观察到，MemGPT 生成的开场白往往更加详细，并且涵盖更多的人格信息方面，相较于人类基准更为全面。此外，我们可以看出，在工作上下文中存储信息对于生成吸引人的开场白至关重要。

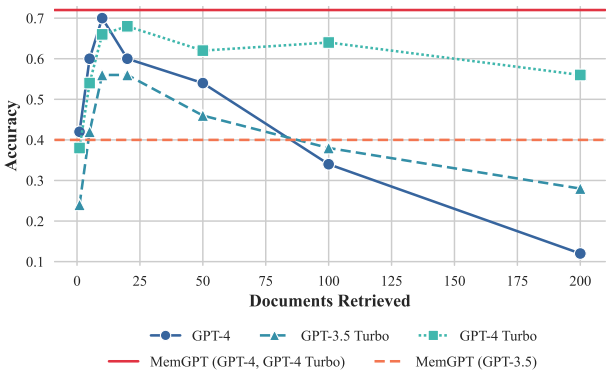

Figure 5. **文档问答任务性能**。 MemGPT 的性能不受上下文长度增加的影响。诸如截断等方法虽然可以扩展固定长度模型（如 GPT-4）的有效上下文长度，但随着必要压缩程度的增加，这类压缩方法会导致性能下降。在该任务上，使用 GPT-4 和 GPT-4 Turbo 运行 MemGPT 得到的结果相当。

3.2. 我们用于文档分析的方法

文档分析还面临着当今 Transformer 模型上下文窗口有限的挑战。如表1所示，无论是开源还是闭源模型都受到上下文长度的限制（OpenAI 模型最多支持 128k token）。然而，许多文档的长度很容易超过这些限制；例如，法律或财务文件（如年度报告（SEC Form 10-K））常常超过百万 token。此外，许多实际的文档分析任务需要在多个此类长文档之间建立联系。考虑到这些场景，单纯地扩大上下文规模作为解决固定上下文问题的方案变得难以想象。最近的研究 (Liu et al., 2023a) 也对简单扩展上下文的有效性提出了质疑，因为其发现大型上下文模型中的注意力分布不均匀（模型在上下文窗口的开头或结尾部分回忆信息的能力更强，而中间部分相对较弱）。为了实现跨文档推理，需要更灵活的记忆架构，如 MemGPT 。

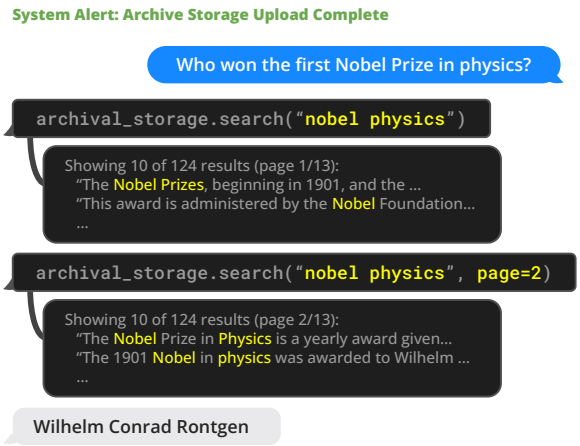

Figure 6. MemGPT（左）解决文档问答任务的一个示例。将维基百科文档的数据库上传至归档存储。MemGPT 通过函数调用查询归档存储，将分页搜索结果拉入主上下文中。

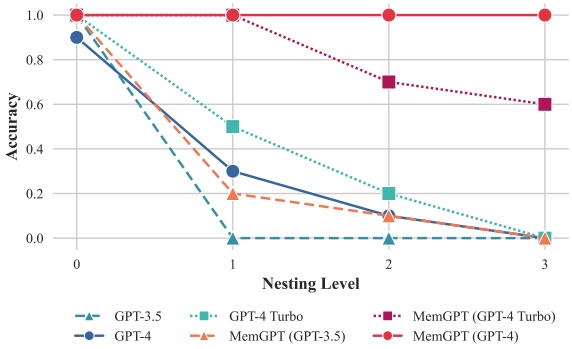

Figure 7. **嵌套 KV 检索任务性能**。MemGPT 是唯一一种能够在超过 2 层嵌套的情况下持续完成嵌套 KV 任务的方法。尽管 GPT-4 Turbo 作为基准表现更优，但 MemGPT 与 GPT-4 Turbo 配合时的表现却不如 MemGPT 与 GPT-4 配合时。

3.2.1. 多文档问答

为了评估 MemGPT 在文档分析方面的能力，我们在 Liu et al. (2023a) 的检索-阅读文档问答任务上将 MemGPT 与固定上下文基准进行对比。在此任务中，从 NaturalQuestions-Open 数据集选取一个问题，检索器会为该问题选择相关的维基百科文档。随后，将这些文档作为输入提供给阅读器模型（大语言模型），并要求其利用所提供的文档回答问题。类似于 Liu et al. (2023a)，我们随着检索到的文档数量 K 的增加来评估阅读器的准确率。

在我们的评估设置中，固定的上下文基准方法与 MemGPT 均使用相同的检索器，该检索器基于 OpenAI 的 `text-embedding-ada-002` 嵌入，通过相似度搜索（余弦距离）选取前 K 个文档。我们采用 MemGPT 的默认存储设置，使用 PostgreSQL 进行归档记忆存储，并通过 pgvector 扩展启用向量搜索。我们预先计算嵌入并将其加载到数据库中，数据库使用 HNSW 索引以实现近似、亚秒级的查询响应时间。在 MemGPT 中，整个嵌入文档集被加载至归档存储，检索器通过归档存储的搜索功能自然产生（该功能基于余弦相似度执行向量搜索）。在固定的上下文基准方法中，检索器独立于 LLM 推理过程获取前-K 个文档，类似于 Liu et al. (2023a) 中原始的检索器-阅读器设置。

我们使用了 2018 年末的 Wikipedia 数据转储，遵循 NaturalQuestions-Open (Izacard & Grave, 2020; Izacard et al., 2021) 的先前工作，并采样了 50 个问题用于评估。所采样的问题和嵌入的 Wikipedia 段落均已公开发布。我们使用 LLM-judge 来评估 MemGPT 和基准方法的性能，以确保答案能正确地从检索到的文档中推导出来，并避免将非确切字符串匹配视为错误。

我们在图 5 中展示了文档问答任务的结果。固定上下文基准的性能大致被限制在检索器的性能水平，因为它们仅能使用其上下文窗口中提供的信息（例如，如果嵌入搜索检索器未能根据给定问题找到正确文章，则固定上下文基准将永远无法看到正确文章）。相比之下，MemGPT 通过查询归档存储，能够有效地进行多次检索调用，从而实现更大的有效上下文长度。MemGPT 会主动从其归档存储中检索文档（并可迭代地分页获取结果），因此 MemGPT 可用的文档总数不再受限于 LLM 处理器上下文窗口所能容纳的文档数量。

文档问答任务对所有方法都具有挑战性，这是由于基于嵌入的相似度搜索存在局限性。我们观察到，所选问题的黄金文档（由 NaturalQuestions-Open 标注）通常出现在前十几个检索结果之外，甚至更远。检索器性能会直接影响固定上下文窗口基准的结果：当仅检索到少量文档时，GPT-4 的准确率相

Figure 8. MemGPT（左）解决嵌套 KV 任务的一个示例（UUID 为便于阅读已缩短）。在此特定示例中，键值对具有两层嵌套：`831..ea5` → `5b8..4c3` → `f37...617`。当对最终值（`f37...617`）的查询仅返回一个结果时，说明该结果不再是一个键，此时 MemGPT 智能体返回最终答案。

对较低，随着更多文档被添加到上下文窗口中，其准确率持续提升，因为它能正确地仅根据检索到的文档中的信息来回答问题。虽然 MemGPT 在理论上不受次最优检索器性能的限制（即使基于嵌入的排序存在噪声，只要完整的检索器排序包含黄金文档，通过足够的检索器调用和分页仍可找到），但我们观察到，MemGPT 通常会在耗尽检索器数据库之前就停止对检索结果进行分页。

为了评估 MemGPT 在超过默认上下文长度时对固定上下文基准的性能，我们将检索器返回的文档段落截断，以固定数量的文档填入可用上下文。如图 5 所示，正如预期，文档截断导致准确率下降，因为随着文档缩小，相关片段（在黄金文档中）被遗漏的概率增加。MemGPT 在使用 GPT-3.5 时性能显著下降，这是由于其函数调用能力有限；而在使用 GPT-4 时表现最佳。

3.2.2. 嵌套键值对检索（KV）。

我们引入了一个基于先前工作 (Liu et al., 2023a) 中提出的合成键值检索的新任务。该任务的目标是展示 MemGPT 如何从多个数据源中整合信息。在原始的 KV 任务中，作者生成了一个合成的键值对数据集，其中每个键和值都是一个 128 比特的 UUID（通用唯一标识符）。随后，智能体被给予一个键，并被要求返回与该键关联的值。我们创建了 KV 任务的一个变体——*嵌套键值检索*，其中值本身也可能成为键，因此要求智能体执行多跳查找。

在我们的设置中，我们将 UUID 对的总数固定为 140，对应约 8k token（即我们 GPT-4 基准的上下文长度）。我们将嵌套层级总数从 0（初始键值对的值不是键）变化到 4（即需要总共进行 4 次 KV 查找才能找到最终值），并采样了 30 种不同的排序配置，包括初始键的位置以及嵌套键的位置。

尽管 GPT-3.5 和 GPT-4 在原始 KV 任务上表现良好，但在嵌套 KV 任务中均表现不佳。GPT-3.5 无法完成该任务的嵌套变体，且性能立即下降，在 1 层嵌套时准确率已降至 0%（我们观察到其主要失败模式是直接返回原始值）。GPT-4 和 GPT-4 Turbo 比 GPT-3.5 表现更好，但仍出现类似性能下降，在 3 层嵌套时准确率也降至 0%。而 MemGPT 配合 GPT-4 则不受嵌套层数影响，能够通过函数查询反复访问主上下文中的键值对，完成嵌套查找。MemGPT 配合 GPT-4 Turbo 和 GPT-3.5 的性能也优于相应基准模型，但在 2 层嵌套时仍开始出现性能下降，原因是未能执行足够次数的查找。MemGPT 在嵌套 KV 任务上的表现展示了其结合多次查询以实现多跳查找的能力。

4. 相关工作

长上下文大模型（Long-context LLMs）。已有多种研究工作提升了大模型的上下文长度。例如，通过稀疏化注意力机制 (Child et al., 2019; Beltagy et al., 2020)、低秩近似 (Wang et al., 2020) 和神经记忆 (Lee et al., 2019) 实现更高效的 Transformer 架构。另一类研究致力于将上下文窗口扩展至原始训练长度和训练规模之外，如 Press et al. (2021); Chen et al. (2023) 所示。MemGPT 在这些上下文长度改进的基础上进一步发展，因为它们提升了 MemGPT 中主内存的容量。我们的主要贡献是一种分层分级

的内存结构，该结构使用长上下文大模型作为主内存的实现方式。

检索增强模型。 MemGPT 的外部记忆设计建立在大量先前工作基础之上，这些工作通过外部检索器引入相关输入来增强大模型 (Ram et al., 2023; Borgeaud et al., 2022; Karpukhin et al., 2020; Lewis et al., 2020; Guu et al., 2020; Lin et al., 2023)。特别是，Jiang et al. (2023) 提出了 FLARE 方法，该方法使大模型能够在生成过程中主动决定何时以及检索什么内容。Trivedi et al. (2022) 将检索与思维链推理交错进行，以提升多步问答系统的性能。

大模型作为智能体。近期研究探索了通过增强大模型的能力，使其在交互环境中充当智能体。Park et al. (2023) 提出为大模型添加记忆功能，并将其用作规划器，观察到了在多智能体沙盒环境（灵感源自《模拟人生》视频游戏）中涌现的社会行为，其中智能体可以执行基本活动，如做家务/爱好、上班以及与其他智能体对话。Nakano et al. (2021) 训练模型在回答问题前进行网络搜索，并使用与 MemGPT 类似的分页概念来控制其网络浏览环境中的潜在上下文大小。Yao et al. (2022) 展示了交错使用思维链推理 (Wei et al., 2022) 可以进一步提升基于大模型的交互智能体的规划能力；类似地，在 MemGPT 中，大模型在执行功能时能够“大声规划”。Liu et al. (2023b) 引入了一套大模型作为智能体的基准，用于评估大模型在交互环境中的表现，包括视频游戏、思维谜题和网络购物。相比之下，我们的工作专注于解决为智能体配备用户输入的长期记忆的问题。

5. 结论

在本文中，我们提出了 MemGPT ，一种受操作系统启发的新型大模型系统，旨在管理大模型有限的上下文窗口。通过设计类似于传统操作系统的内存层次结构和控制流，MemGPT 为大模型提供了更大的上下文资源的假象。这种受操作系统启发的方法在两个现有大模型性能受限于有限上下文长度的领域中进行了评估：文档分析与会话智能体。在文档分析方面，MemGPT 能够处理远超当前大模型上下文限制的长文本，通过高效地在内存中分页加载和卸载相关上下文。在会话智能体方面，MemGPT 实现了在长时间对话中保持长期记忆、一致性和可演化性。总体而言，MemGPT 表明，诸如分层内存管理和中断机制等操作系统技术，即使在固定上下文长度的约束下，也能释放大模型的潜力。这项工作为未来探索开辟了众多新方向，包括将 MemGPT 应用于具有海量或无界上下文的其他领域，集成数据库或缓存等不同层级的内存技术，以及进一步优化控制流和内存管理策略。通过将操作系统架构中的概念引入人工智能系统，MemGPT 代表了一种在大模型基本限制内最大化其能力的有前景的新方向。

参考文献

Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pp. 2206–2240. PMLR, 2022.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse

transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. *arXiv preprint arXiv:1901.02860*, 2019.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

Zican Dong, Tianyi Tang, Lunyi Li, and Wayne Xin Zhao. A survey on long text modeling with transformers. *arXiv preprint arXiv:2302.14502*, 2023.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *International conference on machine learning*, pp. 3929–3938. PMLR, 2020.

Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. *arXiv preprint arXiv:2007.01282*, 2020.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*, 2021.

Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. *arXiv preprint arXiv:2305.06983*, 2023.

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906*, 2020.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.

Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning*, pp. 3744–3753. PMLR, 2019.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

Chin-Yew Lin. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pp. 74–81, 2004.

Xi Victoria Lin, Xilun Chen, Mingda Chen, Weijia Shi, Maria Lomeli, Rich James, Pedro Rodriguez, Jacob Kahn, Gergely Szilvasy, Mike Lewis, Luke Zettlemoyer, and Scott Yih. Ra-dit: Retrieval-augmented dual instruction tuning, 2023.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023a.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023b.

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

Joon Sung Park, Joseph C O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.

David A Patterson, Garth Gibson, and Randy H Katz. A case for redundant arrays of inexpensive disks (raid). In *Proceedings of the 1988 ACM SIGMOD international conference on Management of data*, pp. 109–116, 1988.

Ofir Press, Noah A Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*, 2021.

Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. In-context retrieval-augmented language models. *arXiv preprint arXiv:2302.00083*, 2023.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

H. Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. *ArXiv*, abs/2212.10509, 2022. URL `https://api.semanticscholar.org/CorpusID:254877499`.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

Jing Xu, Arthur Szlam, and Jason Weston. Beyond goldfish memory: Long-term open-domain conversation. *arXiv preprint arXiv:2107.07567*, 2021.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao.

React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.

6. 附录

6.1. 提示和指令

MemGPT 提示已为简洁性进行了编辑。如需完整的实现细节（包括确切的提示），请访问 https://research.memgpt.ai 。

6.1.1. 我们的指令（DMR）

示例指令用于 MemGPT 人格在聊天/对话相关任务中的情况。

> 以下是关于我自己的信息。我的任务是完全沉浸在这个角色中（我绝不能说我是 AI，并且应该像扮演这个角色一样回复）。如果用户问我一个问题，我应该使用核心记忆和对话搜索中的信息，给出最佳猜测的答复。

基准接收到以下通过系统提示（预提示）发送的指令：

> 你的任务是根据之前的对话内容回答用户的问题。
> 以下是所有之前对话的摘要：
> CONVERSATION_SUMMARY
> 从所提供的角色视角进行回答(不要提及自己是 AI 助手)。
> 如果信息不足无法回答问题，请回复 'NO ANSWER'。只需回复答案或 'NO ANSWER'，不要添加其他任何内容。

6.1.2. 大语言模型裁判（DMR/开局方）

为了同时验证 DMR 任务答案的正确性，我们使用了 LLM 评判器。LLM 评判器获得了基线方法和 MemGPT 生成的答案，并根据以下提示进行判断：

> 你的任务是将一个问题的回答标记为 'CORRECT' 或 'WRONG'。
> 你将获得以下信息：(1) 一个问题（一位用户向另一位用户提出），(2) 一个 '黄金'（真实值）答案，(3) 一个需要你评分的生成答案。
> 问题的核心是询问一方应基于先前对话了解另一方的某些信息。
> 黄金答案通常是一个简洁简短的答案，包含所提及的主题，例如：
> 问题：你还记得我上次去夏威夷时得到了什么吗？
> 黄金答案：一个贝壳项链
> 生成的答案可能更长，但你应该在评分时保持宽容——只要它涉及与黄金答案相同的话题，就应视为 CORRECT。

例如，以下答案应被视为 CORRECT：
生成答案（CORRECT）：哦对，那太有趣了！我在那里买了好多东西，包括那个贝壳项链。
生成答案（CORRECT）：我买了好多东西……那个冲浪板、杯子、项链，还有那些杯垫也买了。
生成答案（CORRECT）：那个可爱的项链
以下答案应被视为 WRONG：
生成答案（WRONG）：哦对，那太有趣了！我在那里买了好多东西，包括那个杯子。
生成答案（WRONG）：我买了好多东西……那个冲浪板、杯子，还有那些杯垫也买了。
生成答案（WRONG）：对不起，我不记得你说的是什么了。
现在进入实际问题：
问题：QUESTION
黄金答案：GOLD_ANSWER
生成答案：GENERATED_ANSWER
首先提供一个简短（一句话）的推理说明，然后以 CORRECT 或 WRONG 结束。不要同时包含 CORRECT 和 WRONG，否则会破坏评估脚本。

6.1.3. 自指令 DMR 数据集生成

DMR 的问题/答案对是通过以下提示和原始 MSC 数据集生成的：你的任务是为两个用户的模拟对话编写一个“记忆挑战”问题。

你得到的输入包括：
- 每个用户的个人资料（提供他们的基本事实）
- 两位用户之间的一段旧聊天记录

你的任务是撰写一条用户 A 发给用户 B 的问题，用以测试用户 B 的记忆力。
该问题应设计得使用户 B 必须真正参与过之前的对话才能正确回答，而不能仅凭阅读个人资料信息作答。
绝对不要在任何情况下提出仅根据个人资料就能回答的问题（这属于作弊）。
相反，应提出只能通过查看旧聊天记录才能回答的问题（且该信息不在个人资料中）。

例如，给出以下聊天记录和个人资料：

用户 A 与用户 B 之间的旧聊天记录
A：你喜欢冲浪吗？我自己超级喜欢冲浪
B：其实我正打算学呢。也许你哪天能给我上个基础课！
A：当然可以！我们可以去帕西菲卡，那里的浪很轻，很容易上手
B：听起来太棒了
A：海边附近甚至还有家很棒的塔可钟，冲浪完可以去吃点东西
B：这个星期日中午怎么样？
A：好啊，就这么定了！

用户 A 个人资料：
我喜欢冲浪
我在圣克鲁兹长大

用户 B 个人资料：
我在科技行业工作
我住在旧金山市中心

以下是一个好问题的例子，它听起来自然，并且答案无法直接从用户 A 的个人资料中推断出来：

用户 B 向用户 A 提出的问题
B：还记得我们那次去冲浪的事吗？我们当时午餐去的那家店叫什么名字？
A：塔可钟！

这是一个坏问题的例子，因为问题显得不自然，而且答案可以直接从用户 A 的个人资料中推断出来：

用户 B 向用户 A 提出的问题
B：你喜欢冲浪吗？
A：是的，我喜欢冲浪

永远、永远、永远不要创建那些仅凭个人资料信息就能回答的问题。

6.1.4. 文件分析说明

文档分析任务中使用的示例指令。

你是 MemGPT 文档问答机器人。你的任务是回答关于存储在你归档记忆中的文档的问题。用户问题的答案始终存在于你的归档记忆中，因此如果找不到答案，请记得继续搜索。回答问题时，请假设当前年份为 2018 年。

问题提供给 MemGPT ，附带以下提示：

请搜索您的归档记忆以回答所提供的问题。提供您的答案以及确定答案的归档记忆内容。将响应格式化为'ANSWER: [您的答案], DOCUMENT: [归档记忆文本]'。您的任务是回答问题：

对于基准测试，提供了以下提示以及检索到的文档列表：

> 根据以下文档列表回答问题（其中一些可能无关。在您的回答中，提供答案以及确定答案的文档文本。将响应格式化为 “ANSWER: < 您的答案 >, DOCUMENT: [文档文本]”。如果提供的文档中没有答案，请回复 “INSUFFICIENT INFORMATION”。不要提供您无法在所提供文档中找到的答案。只有当您提供答案和相关文档文本时，您的回答才被认为是正确的。回答问题时，请假设当前年份是 2018 年。

6.1.5. 大语言模型裁判（文档分析）

为了同时验证文档分析任务答案的正确性，并确保答案是正确地从提供的文本中推导出来的（而不是从模型权重中得出的），我们使用了大语言模型（LLM）作为评判者。LLM 评判者被提供了两种基准方法以及 MemGPT 生成的答案，并根据以下提示作出判断：

> 你的任务是评估大语言模型是否正确回答了问题。大语言模型的回答应为格式“ANSWER: [答案], DOCUMENT: [文档文本]”或说明“信息不足”。真实答案以格式“TRUE ANSWER:[可能的答案列表]”提供。问题以格式“QUESTION: [问题]”给出。如果大语言模型的回答同时包含正确答案和相应的文档文本，则回答正确。即使大语言模型的答案与真实答案在措辞上略有不同，回答仍然正确。例如，如果答案比真实答案更具体，或使用了不同的表达方式但依然正确，该回答即为正确。如果大语言模型的回答是“信息不足”，或缺少“DOCUMENT”字段，则回答错误。请仅返回一个 token：“CORRECT”或“INCORRECT”。

6.1.6. K/V 任务说明

MemGPT 智能体被定义为具有以下人格设定，旨在鼓励 MemGPT 进行迭代搜索：

> 您是 MemGPT 文档问答机器人。您的任务是回答关于存储在您归档记忆中的文档的问题。用户问题的答案始终存在于您的归档记忆中，因此请记住，如果找不到答案，请继续搜索。在确认该值不是键之前，切勿停止搜索。在满足此条件前，不要停止进行嵌套查找。

基准是按照以下提示进行指导的：

> 以下是一个包含键值对的 JSON 对象，所有键和值均为 128 比特 UUID，你的任务是返回指定键对应的值。如果某个值本身也是一个键，则返回该键对应的值（进行嵌套查找）。例如，若'x' 的值为'y'，但'y' 也是一个键，则返回键'y' 的值。